

Part I: Object-Oriented Programming Techniques

1.1 Polymorphism and Virtual Functions

Concept: Polymorphism

Polymorphism allows objects of different classes to be treated through a common interface. In C++, this is achieved using **virtual functions**.

- A **virtual function** is declared with the **virtual** keyword in the base class.
- A **pure virtual function** (e.g., `virtual void f() = 0;`) makes the class *abstract* — it cannot be instantiated.
- Derived classes **override** virtual functions to provide specific behaviour.
- Calling a virtual function through a *base-class pointer* invokes the correct derived-class version at runtime (**dynamic dispatch**).

Example 1 — Virtual Function (Shape Hierarchy)

```
#include <iostream>
#include <cmath>
using namespace std;

class Shape {
public:
    virtual double area() = 0;    // pure virtual function
    virtual ~Shape() {}
};

class Circle : public Shape {
    double radius;
public:
    Circle(double r) : radius(r) {}
    double area() override { return M_PI * radius * radius; }
};

class Rectangle : public Shape {
    double length, width;
public:
    Rectangle(double l, double w) : length(l), width(w) {}
    double area() override { return length * width; }
};

class Triangle : public Shape {
    double base, height;
public:
    Triangle(double b, double h) : base(b), height(h) {}
};
```

```

    double area() override { return 0.5 * base * height; }
};

int main() {
    Shape* shapes[] = { new Circle(5.5),
                        new Rectangle(8, 4.2),
                        new Triangle(6, 3.5) };
    const char* names[] = { "Circle", "Rectangle", "Triangle" };
    for (int i = 0; i < 3; i++)
        cout << names[i] << " area: " << shapes[i]->area() << "\n";
    for (auto s : shapes) delete s;
}

```

Expected output:

Circle area: 95.0332

Rectangle area: 33.6

Triangle area: 10.5

Example 2 — Pure Virtual Functions (Banking System)

```

#include <iostream>
using namespace std;

class Account {
protected:
    double balance, annualRate;
    int time;
public:
    Account(double b, double r, int t) : balance(b), annualRate(r), time(t) {}
    virtual double calculateInterest() = 0;
    virtual ~Account() {}
};

class SavingsAccount : public Account {
public:
    SavingsAccount(double b, double r, int t) : Account(b, r, t) {}
    double calculateInterest() override {
        return balance * annualRate * (time / 12.0);
    }
};

class CheckingAccount : public Account {
    double monthlyFee;
public:
    CheckingAccount(double b, double r, int t, double fee)
        : Account(b, r, t), monthlyFee(fee) {}
    double calculateInterest() override {
        return (balance - monthlyFee) * annualRate * (time / 12.0);
    }
};

```

```
int main() {
    Account* acc1 = new SavingsAccount(5000, 0.03, 6);
    Account* acc2 = new CheckingAccount(3000, 0.015, 6, 10);
    cout << fixed;
    cout.precision(2);
    cout << "Savings Account Interest: $" << acc1->calculateInterest() << "\n";
    cout << "Checking Account Interest: $" << acc2->calculateInterest() << "\n";
    delete acc1; delete acc2;
}
```

Expected output:

Savings Account Interest: \$75.00

Checking Account Interest: \$21.75

1.2 Inheritance and the Diamond Problem

Concept: Inheritance Types

- **Single Inheritance:** a class derives from one base class.
- **Multiple Inheritance:** a class derives from more than one base class.
- **Hybrid Inheritance:** a combination of multiple inheritance types.
- **Diamond Problem:** when two classes B and C both inherit from A, and class D inherits from both B and C, there are two copies of A in D by default.
- **Solution — Virtual Inheritance:** declaring the base class as `virtual` ensures only one shared copy exists.

Example 3 — Virtual Inheritance (Teaching Assistant)

```
#include <iostream>
#include <stdexcept>
using namespace std;

class Person {
protected:
    string name;
    int age;
public:
    Person(const string& n, int a) : name(n), age(a) {
        if (a < 0) throw invalid_argument("Age cannot be negative.");
        cout << "Person constructor called\n";
    }
    virtual void display() {
        cout << "Name: " << name << "\nAge: " << age << "\n";
    }
    virtual string getRole() { return "Person"; }
    virtual ~Person() {}
};
```

```

class Student : virtual public Person {
    string major;
public:
    Student(const string& n, int a, const string& m)
        : Person(n, a), major(m) {
        cout << "Student constructor called\n";
    }
    string getRole() override { return "Student"; }
};

class Employee : virtual public Person {
    string department;
public:
    Employee(const string& n, int a, const string& d)
        : Person(n, a), department(d) {
        cout << "Employee constructor called\n";
    }
    string getRole() override { return "Employee"; }
};

class TeachingAssistant : public Student, public Employee {
public:
    TeachingAssistant(const string& n, int a,
                     const string& m, const string& d)
        : Person(n, a), Student(n, a, m), Employee(n, a, d) {
        cout << "TeachingAssistant constructor called\n";
    }
    string getRole() override { return "Teaching Assistant"; }
    void display() override {
        cout << "Role: " << getRole() << "\n";
        Person::display();
    }
};

void testDiamondProblem() {
    cout << "\nTesting Diamond Problem...\n";
    cout << "Base class Person appears only once "
          << "due to virtual inheritance.\n";
}

int main() {
    cout << "Creating a Teaching Assistant...\n";
    TeachingAssistant ta("Alice", 25, "CS", "Engineering");
    cout << "\n";
    ta.display();
    testDiamondProblem();

    Person* p = &ta;
    cout << "\nCalling display() via base pointer:\n";
    p->display();
}

```

Expected output:

Creating a Teaching Assistant...

Person constructor called

Student constructor called

Employee constructor called

TeachingAssistant constructor called

Role: Teaching Assistant

Name: Alice

Age: 25

Testing Diamond Problem...

Base class Person appears only once due to virtual inheritance.

1.3 Exception Handling

Concept: Exception Handling

C++ provides a structured mechanism to handle runtime errors:

- **try** — encloses the code that may throw an exception.
- **throw** — raises an exception object.
- **catch** — handles a specific type of exception.
- **Custom exceptions** are created by inheriting from `std::exception` and overriding `what()`.

Example 4 — Custom Exception (Safe Division)

```
#include <iostream>
#include <stdexcept>
#include <cmath>
using namespace std;

class DivisionByZeroException : public runtime_error {
public:
    DivisionByZeroException()
        : runtime_error("Division by zero is not allowed.") {}
};

double safeDivide(double num, double den) {
    if (den == 0) throw DivisionByZeroException();
    if (den < 0) {
        cout << "Warning: Converting negative denominator to positive.\n";
        den = abs(den);
    }
    return num / den;
}
```

```
int main() {
    double num, den;
    char again = 'y';
    while (again == 'y' || again == 'Y') {
        cout << "Enter numerator and denominator: ";
        if (!(cin >> num >> den)) {
            cout << "Invalid input! Please enter numerical values.\n";
            cin.clear();
            cin.ignore(1000, '\n');
        } else {
            try {
                cout << "Result: " << safeDivide(num, den) << "\n";
            } catch (const DivisionByZeroException& e) {
                cout << "Error: " << e.what() << "\n";
            }
        }
        cout << "Try again? (y/n): ";
        cin >> again;
    }
}
```

Part II: Generic Programming with Templates

2.1 Function Templates

Concept: Function Templates

A **function template** lets you write a single function that works for any data type. The compiler generates type-specific copies at compile time (this is called *template instantiation*).

```
template <typename T>
T functionName(T a, T b) { /* ... */ }
```

- **typename** and **class** are interchangeable in template parameter lists.
- The compiler deduces the type from the arguments; explicit specialisation (`functionName<int>(...)`) is also possible.
- Advantages: **code reuse**, avoids function overloading, type safety.

Example 5 — Generic Max, Sum, and Swap

```
#include <iostream>
using namespace std;

template <typename T>
T my_max(T a, T b) { return (a > b) ? a : b; }

template <typename T>
T my_sum(T a, T b) { return a + b; }

template <typename T>
void swap_values(T& a, T& b) { T tmp = a; a = b; b = tmp; }

int main() {
    cout << my_max(4, 5) << "\n";
    cout << my_max(4.5, 2.5) << "\n";
    cout << my_max('a', 'b') << "\n";

    cout << my_sum(4, 5) << "\n";
    cout << my_sum(string("IBA "), string("Karachi")) << "\n";

    int x = 10, y = 20;
    swap_values(x, y);
    cout << "After swap: x=" << x << " y=" << y << "\n";
}
```

2.2 Class Templates

Concept: Class Templates

A **class template** parameterises an entire class over one or more types (and optionally non-type parameters such as `int size`).

```
template <class T, int N>
class MyArray { T data[N]; /* ... */ };
```

Instantiation example: `MyArray<int, 5> arr;`

Example 6 — Generic Array Class

```
#include <iostream>
using namespace std;

template <class T, int size>
class Array {
    T arr[size];
public:
    T& operator[](int index) { return arr[index]; }
};

int main() {
    Array<int, 3> obj;
    Array<double, 5> obj2;

    for (int i = 0; i < 3; i++) {
        obj[i] = i * 2;
        cout << "intArray[" << i << "] = " << obj[i] << "\n";
    }
    for (int i = 0; i < 5; i++) {
        obj2[i] = i + 2.5;
        cout << "doubleArray[" << i << "] = " << obj2[i] << "\n";
    }
}
```

Example 7 — Generic Stack Class

```
#include <iostream>
#include <vector>
#include <stdexcept>
using namespace std;

template <typename T>
class Stack {
    vector<T> data;
public:
    void push(const T& val) { data.push_back(val); }
    void pop() {
```



```

        if (data.empty()) throw underflow_error("Stack is empty.");
        data.pop_back();
    }
    T& top() {
        if (data.empty()) throw underflow_error("Stack is empty.");
        return data.back();
    }
    bool empty() const { return data.empty(); }
};

int main() {
    Stack<int> s;
    s.push(10); s.push(20); s.push(30);
    cout << "Top: " << s.top() << "\n";
    s.pop();
    cout << "After pop, top: " << s.top() << "\n";
}

```

Example 8 — Generic Queue Class

```

#include <iostream>
#include <deque>
#include <stdexcept>
using namespace std;

template <typename T>
class Queue {
    deque<T> data;
public:
    void enqueue(const T& val) { data.push_back(val); }
    void dequeue() {
        if (data.empty()) throw underflow_error("Queue is empty.");
        data.pop_front();
    }
    T& front() {
        if (data.empty()) throw underflow_error("Queue is empty.");
        return data.front();
    }
    bool empty() const { return data.empty(); }
};

int main() {
    Queue<string> q;
    q.enqueue("first"); q.enqueue("second"); q.enqueue("third");
    cout << "Front: " << q.front() << "\n";
    q.dequeue();
    cout << "After dequeue, front: " << q.front() << "\n";
}

```

Example 9 — Generic Linked List

```
#include <iostream>
using namespace std;

template <typename T>
class LinkedList {
    struct Node { T data; Node* next; };
    Node* head = nullptr;
public:
    void insert(const T& val) {
        Node* n = new Node{val, head};
        head = n;
    }
    void remove(const T& val) {
        Node** cur = &head;
        while (*cur) {
            if ((*cur)->data == val) {
                Node* tmp = *cur;
                *cur = (*cur)->next;
                delete tmp;
                return;
            }
            cur = &(*cur)->next;
        }
    }
    void traverse() const {
        for (Node* cur = head; cur; cur = cur->next)
            cout << cur->data << " -> ";
        cout << "nullptr\n";
    }
    ~LinkedList() {
        while (head) { Node* tmp = head; head = head->next; delete tmp; }
    }
};

int main() {
    LinkedList<int> list;
    list.insert(3); list.insert(2); list.insert(1);
    list.traverse();    // 1 -> 2 -> 3 -> nullptr
    list.remove(2);
    list.traverse();    // 1 -> 3 -> nullptr
}
```

Lab Exercises

Exercise 1

[**Polymorphism**] Implement a base class **Shape** with a pure virtual function **area()**. Derive **Circle**, **Rectangle**, and **Triangle** from it.

- Circle: Radius = 5.5 (Area = πr^2)
- Rectangle: Length = 8, Width = 4.2 (Area = $l \times w$)
- Triangle: Base = 6, Height = 3.5 (Area = $\frac{1}{2}bh$)

Create objects of each class and display their areas via base-class pointers.

Exercise 2

[**Abstract Classes**] Create a polymorphic banking system with a base class **Account** containing a pure virtual function **calculateInterest()**. Derive **SavingsAccount** and **CheckingAccount**:

- **Savings:** Interest = $\text{balance} \times \text{rate} \times \frac{\text{time}}{12}$
- **Checking:** Interest = $(\text{balance} - \text{fee}) \times \text{rate} \times \frac{\text{time}}{12}$

Use: Savings (balance=\$5000, rate=3%, time=6 months); Checking (balance=\$3000, rate=1.5%, time=6 months, fee=\$10).

Expected output:

Savings Account Interest: \$75.00

Checking Account Interest: \$21.75

Exercise 3

[**Exception Handling — Division**] Implement a robust division function with:

- A custom exception **DivisionByZeroException** for zero denominators.
- Graceful handling of non-numeric input.
- Conversion of negative denominators to positive with a warning.
- A retry loop allowing multiple operations.

Exercise 4

[**Exception Handling — Files**] Implement a class **FileHandler** that manages open, read, write, and close operations on files, with proper exception handling for file-not-found and permission errors.

Exercise 5

[**Hybrid Inheritance & Diamond Problem**] Design the following hierarchy using **virtual inheritance**:

- Person (base): name, age, display(), getRole()
- Student and Employee inherit *virtually* from Person.
- TeachingAssistant inherits from both Student and Employee.

Demonstrate: constructor execution order, exception for negative age, polymorphism via base pointer, and a `testDiamondProblem()` function.

- Exercise 6**
 [Polymorphic Logging] Design a logging system where `ConsoleLogger`, `FileLogger`, and `DatabaseLogger` each inherit from a base class and override a virtual function `logMessage()`.
- Exercise 7**
 [Exception-Safe Matrix] Implement a matrix class with proper memory management; handle invalid operations, out-of-bounds access, and incompatible dimensions via exceptions.
- Exercise 8**
 [Multiple Inheritance] Implement a `Car` class that inherits from both `Vehicle` and `Engine`, where both inherit from `PoweredSystem`. Solve the diamond problem with virtual inheritance.
- Exercise 9**
 [Function Template — Max] Write a generic function `max_value()` that takes three arguments of any type and returns the maximum.
- Exercise 10**
 [Function Template — Swap] Write a generic function `swap_values()` that takes two arguments of any type and swaps them in-place.
- Exercise 11**
 [Function Template — Frequency] Write a generic function `element_frequency()` that takes an array of any type, its size, and a search value, and returns the number of occurrences.
- Exercise 12**
 [Sorting & Searching] Write the following generic functions operating on arrays of any type:
- `sort_array()` — sort in ascending order.
 - `reverse_array()` — reverse the sorted array.
 - `binary_search()` — return the index of a target value (or `-1` if not found).
- Exercise 13**
 [Class Template — Stack] Implement a generic `Stack<T>` class with `push()`, `pop()`, and `top()` operations. Throw an appropriate exception when the stack is empty.
- Exercise 14**
 [Class Template — Queue] Implement a generic `Queue<T>` class with `enqueue()`, `dequeue()`, and `front()` operations.
- Exercise 15**
 [Class Template — LinkedList] Implement a generic `LinkedList<T>` class with insert, delete, and traversal operations.
- Exercise 16**
 [Class Template — SparseMatrix] Implement a generic `SparseMatrix<T>` class supporting addition, subtraction, and multiplication.

Exercise 17

[Higher-Order Templates] Write a generic function `matrix_operation()` that applies a binary operation element-wise to two 2D matrices. Use the following signature:

```
template <typename T, typename BinaryOperation>
vector<vector<T>>> matrix_operation(
    const vector<vector<T>>& m1,
    const vector<vector<T>>& m2,
    size_t rows, size_t cols,
    BinaryOperation op);
```

Demonstrate addition using a lambda: `auto add = [](const T& a, const T& b){ return a+b; };`

Exercise 18

[Class Template — PriorityQueue] Implement a generic `PriorityQueue<T>` class using a comparator template parameter:

```
template <typename T, typename PriorityFunc = std::less<T>>
class PriorityQueue {
public:
    void insert(const T& element);
    T getHighestPriority();
private:
    std::priority_queue<T, std::vector<T>, PriorityFunc> pq;
};
```

Demonstrate with integers and a custom priority function.

Exercise 19

[Singleton] Implement a thread-safe singleton class using C++.

Exercise 20

[Memory Pool] Implement a memory pool allocator in C++ and compare its performance with standard `new` and `delete`.
